

QMST5336: ANALYTICS

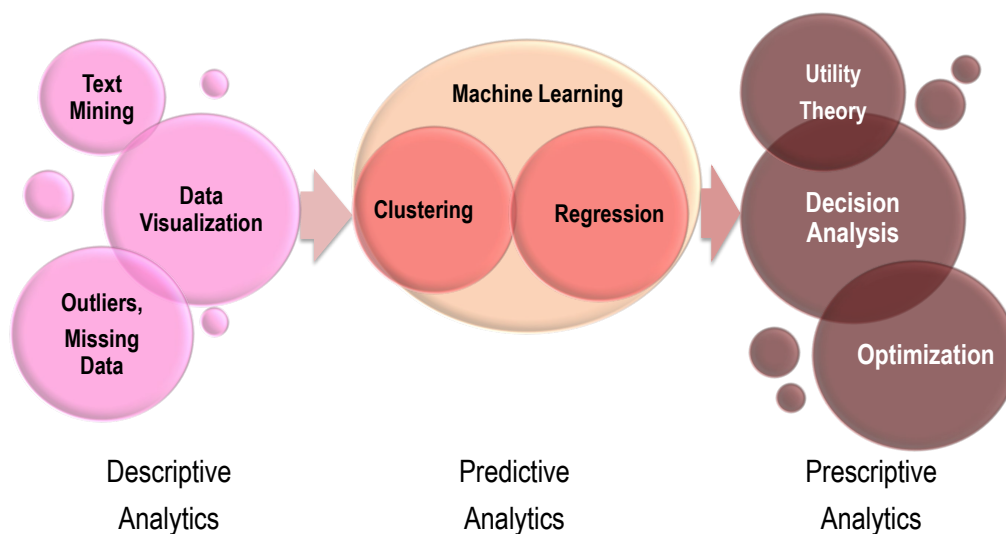
Lecture 3: Detect Outliers & Handle Missing Data

Emily Zhu, PhD.

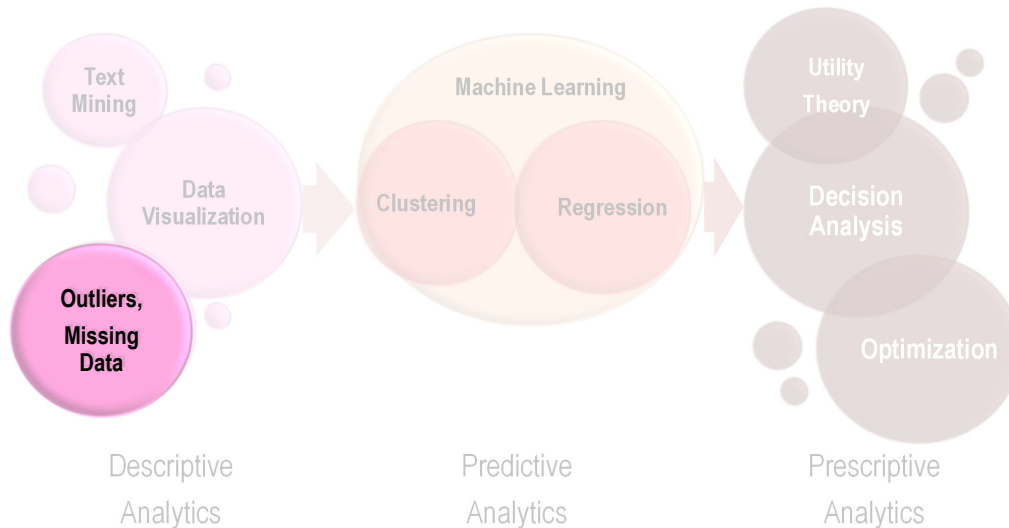
© 2023 Texas State University

Our Problem and Roadmap

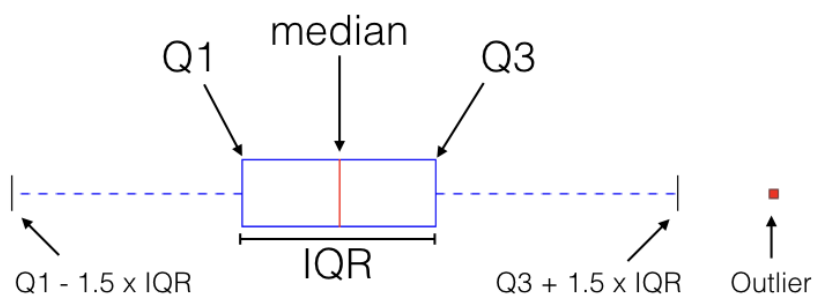
An investment property in Austin: **Airbnb** versus **Rental**?



Roadmap: Where We Are



Outlier Detection: Tukey IQR



Q1: *Quartile 1*, or median of the *left* data subset after dividing the original data set into 2 subsets via the median (25% of the data points fall below this threshold)

Q3: *Quartile 3*, median of the *right* data subset (75% of the data points fall below this threshold)

IQR: *Interquartile-range*, $Q3 - Q1$

Outliers: Data points are considered to be outliers if
 $value < Q1 - 1.5 \times IQR$ or
 $value > Q3 + 1.5 \times IQR$



Sebastian Raschka, 2016
This work is licensed under a Creative Commons Attribution 4.0 International License

Identifies extreme values in data.

The Tukey IQR Function

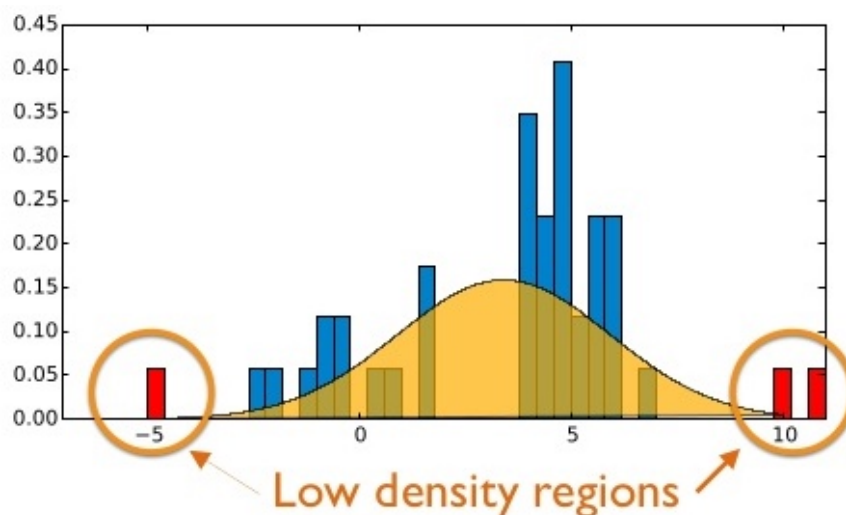
```
1 #----Tukey IQR function-----
2 import numpy as np
3
4 def find_outliers_tukey(x):
5     q1 = np.percentile(x, 25)
6     q3 = np.percentile(x, 75)
7     iqr = q3-q1
8     floor = q1 - 1.5*iqr
9     ceiling = q3 + 1.5*iqr
10    outlier_indices = list(x.index[(x < floor)|(x > ceiling)])
11    outlier_values = list(x[outlier_indices])
12
13    return outlier_indices, outlier_values
14 #----End of the function-----
15
16 #apply the function on the DataFrame df_price
17 tukey_indices, tukey_values = find_outliers_tukey(df_price['price'
    ])

```

```
1 #Remove outliers: 2 equivalent ways
2 df_new1 = df_old[~df_old['column name'].isin(tukey_values)]
3 df_new2 = df_old[~df_old.index.isin(tukey_indices)]

```

Outlier Detection: Kernel Density Estimation



- Non-parametric way to estimate the probability density function of a given feature
- Can be advantageous compared to extreme value detection (e.g. Tukey IQR), as this method captures outliers in bimodal distributions.

The KDE Function

```
1 #----Kernel Density Estimation function-----
2 from sklearn.preprocessing import scale
3 from statsmodels.nonparametric.kde import KDEUnivariate
4
5 def find_outliers_kde(x):
6     x_scaled = scale(list(map(float, x)))
7     kde = KDEUnivariate(x_scaled)
8     kde.fit(bw="scott", fft=True)
9     pred = kde.evaluate(x_scaled)
10
11     n = sum(pred < 0.05)
12     outlier_ind = np.asarray(pred).argsort()[:n]
13     outlier_value = np.asarray(x)[outlier_ind]
14
15     return outlier_ind, outlier_value
16 #----End of the function----
17
18 #apply the function on the DataFrame df_price
19 kde_indices, kde_values = find_outliers_kde(df_price['price'])
20
21 #Remove outliers: 2 equivalent ways
22 df_new1 = df_old[~df_old['column name'].isin(kde_values)]
23 df_new2 = df_old[~df_old.index.isin(kde_indices)]
```

Applications of Outlier Detection

- Fraud detection: abnormal buying patterns can imply credit card abuse
- Medicine: unusual test results
- Sports: identification of outstanding players.



Handle Missing Data



Why Is Data Missing?

The treatment of missing data depends on why the data are missing.

- **Missing Completely At Random (MCAR):** the missing value is truly a random occurrence.
- **Missing At Random (MAR):** the occurrence of some missing values might be correlated with the values of some other variables, but mainly due to data entry error.
- **Missing Not At Random (MNAR):** the reason that the value is missing is related to the value of the variable.

Handle Missing Data

Postcode	Type	# bedroom	# bath	squareft	garage	land (arces)	HOA \$/month	year of construction	# Strokes	Price 000\$	
78759	Condo		2	2	869	0	0.03	2011	1982	2	213
78759	Condo		2	1	979	1	0.13	2088	1985	1	289
78759	Condo		3	3	1616	2	0.07	0	1979	2	269
78759	House		3	2	1380	2	0.22	0	1981	1	390
78759	House		4	2.5	1914	2	0.2	0	1981	2	499
78759	House				2531	2	0.3	0	1975	2	605
78759	House		3	2	1363	1	0.13	0	1978	1	420
78759	Condo		3	2.5	1678	2	0.07	175	1972	2	524
78759	House		3	2	1392	2	0.18	0	1977	1	384.9

Remove this variable if

- a variable is missing for a large number of observations;
- if the variable to be dropped is highly correlated with another variable that is known for a majority of observations, so the loss of information may be minimal.

Delete these incomplete observations if

- the number of observations with missing values is small &
- the missing values seem to be MCAR.

Missing Data Imputation

Replace missing values with the variable's mode, mean or median if

- a considerable number of observations have missing values &
- the missing values are MCAR.

Build models to impute missing values if

- missing values are troublesome &
- the missing values are MAR.

Handle Missing Data in Python

Mark missing data NumPy.nan or Pandas.null

Simple delete Pandas.dropna()

axis : {0 or 'index', 1 or 'columns'}, default 0

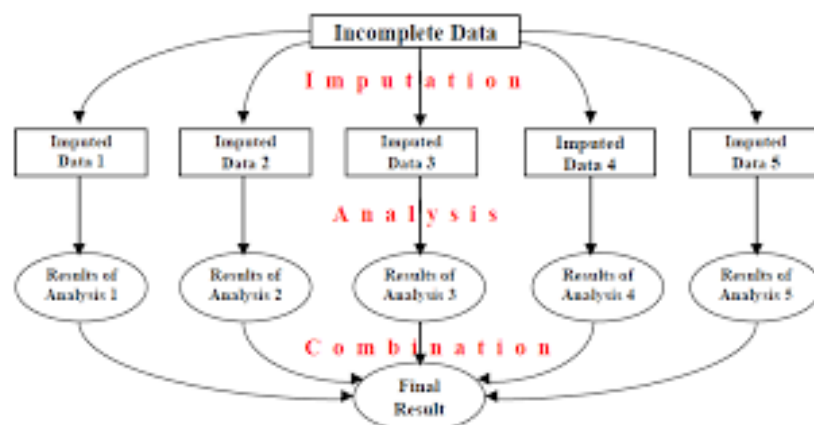
Replacement Pandas.fillna

specify value or method for filling

More advanced statsmodels.imputation or sklearn.impute packages, e.g.

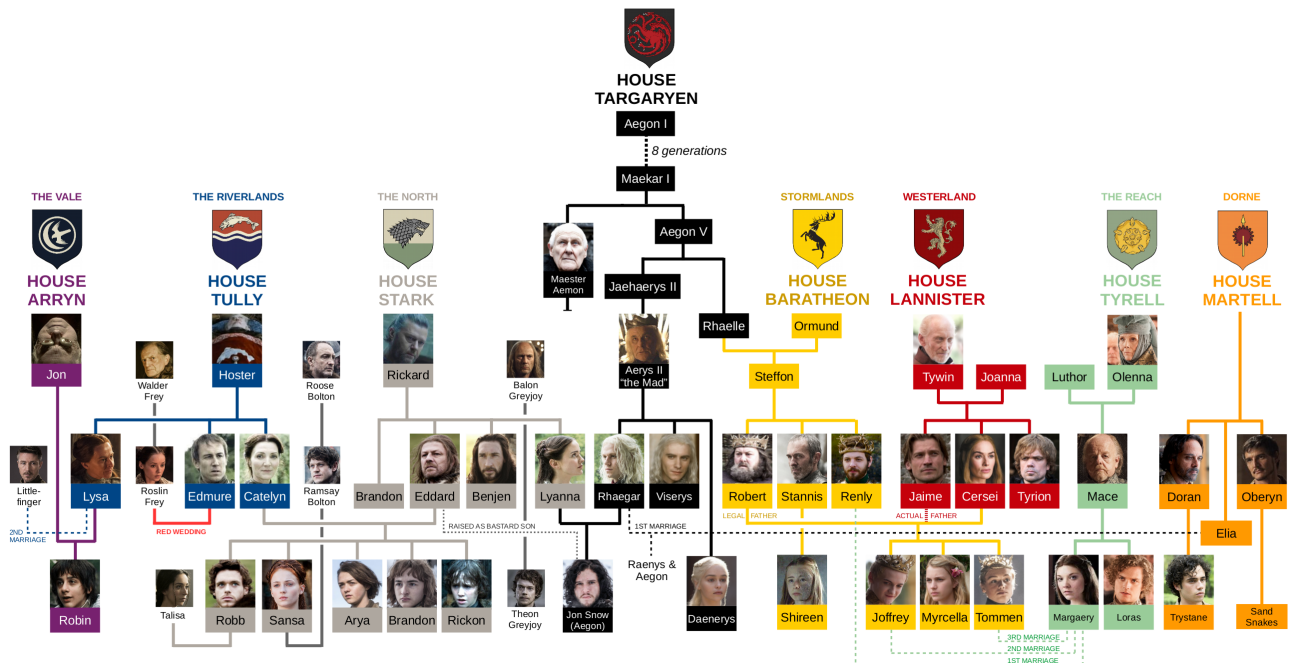
```
1 from statsmodels.imputation import mice
2 imputed_data = mice(raw_data)
```

Multiple Imputations by Chained Equations



shutterstock.com · 98210171

Example: Handle Missing Data



Game of Thrones Family Tree by Matt Baker - UsefulCharts.com. For educational purposes only - NOT FOR SALE

Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

15 / 30

Simple Imputation in Pandas

	Character	Mother	Father	Kills
0	Arya Stark	Catelyn	Eddard	1278
1	Cersei Lannister	Joana	Tywin	199
2	Jon Snow	Lyanna	Rhaegar	112
3	Daenerys Targaryen	None	Aerys II	1699

```
1 df_GoT.dropna()
```

	Character	Mother	Father	Kills
0	Arya Stark	Catelyn	Eddard	1278
1	Cersei Lannister	Joana	Tywin	199
2	Jon Snow	Lyanna	Rhaegar	112

```
1 df_GoT.dropna(axis = 1)
```

	Character	Father	Kills
0	Arya Stark	Eddard	1278
1	Cersei Lannister	Tywin	199
2	Jon Snow	Rhaegar	112
3	Daenerys Targaryen	Aerys II	1699

<https://www.washingtonpost.com/graphics/entertainment/game-of-thrones/>

Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

16 / 30

Simple Imputation in Pandas C'td

```
1 df_GoT.fillna(value = 0, inplace = True)
```

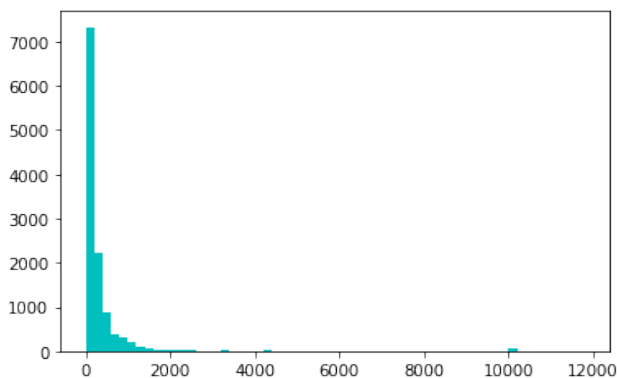
	Character	Mother	Father	Kills
0	Arya Stark	Catelyn	Eddard	1278
1	Cersei Lannister	Joana	Tywin	199
2	Jon Snow	Lyanna	Rhaegar	112
3	Daenerys Targaryen	0	Aerys II	1699

```
1 df_GoT.fillna(method = ffill, inplace = True)
```

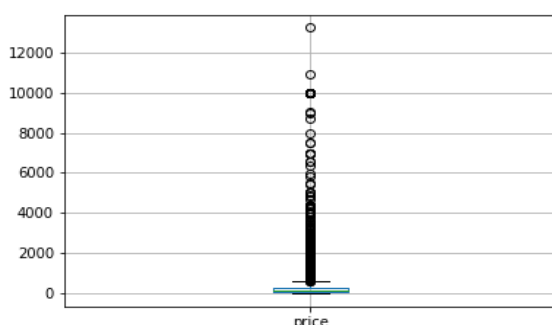
	Character	Mother	Father	Kills
0	Arya Stark	Catelyn	Eddard	1278
1	Cersei Lannister	Joana	Tywin	199
2	Jon Snow	Lyanna	Rhaegar	112
3	Daenerys Targaryen	Lyanna	Aerys II	1699

Case Study: Daily Prices

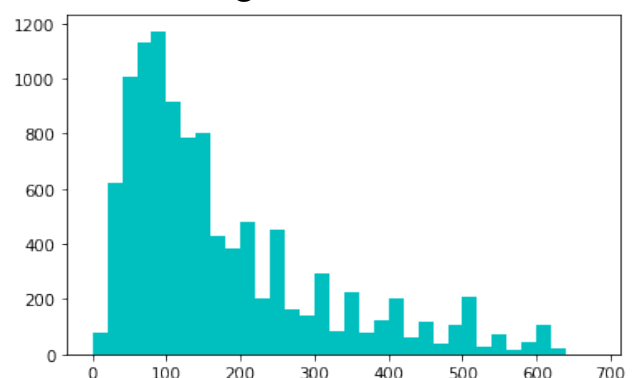
Raw Data



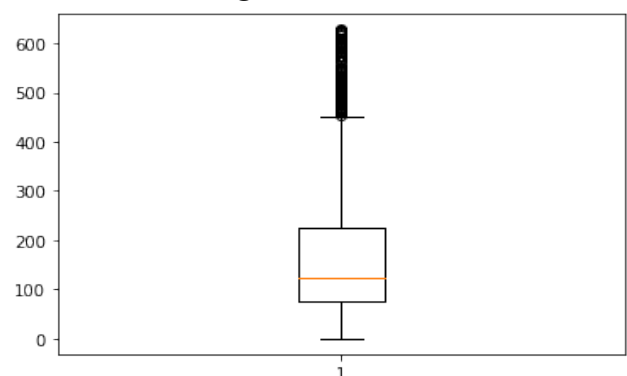
Raw Data



After Removing Outliers



After Removing Outliers



Case: the Price Column

```
1 df_full[[5, 173, 215, 400], 'price']
```

```
5          $99.00
173       $2,000.00
215       $1,000.00
400       $1,300.00
Name: price, dtype: object
```

```
1 df_full.price.str.startswith('$').sum()
```

```
11792
```

- \$ signs
- thousands operators
- convert to numbers

Remove the \$ Sign

```
1 #Method 1:
2 price_1 =
3     df_full.price.str.slice(1,)
4 price_1[:5]
```

```
0      85.00
1     225.00
2     100.00
3      95.00
4      40.00
Name: price, dtype: object
```

```
1 #Method 2:
2 price_2 = df_full.price.str[1:]
3 price_2.equals(price_1)
```

```
True
```

```
1 #Method 3
2 price_3 = df_full.price.str.
3     replace("$", "")
4 price_3.equals(price_2)
```

```
True
```

```
1 #Method 4
2 price_4 = df_full.price.str.
3     lstrip('$')
4 price_4.equals(price_1)
```

```
True
```

Remove Thousands Operators

```
1 price_1[[400, 215]]  
  
400      1,300.00  
215      1,000.00  
Name: price, dtype: object
```

```
1 price_1 = price_1.str.replace(',', '', '')  
2 price_1[[5, 173, 215, 400]]  
  
5          99.00  
173       2000.00  
215       1000.00  
400       1300.00  
Name: price, dtype: object
```

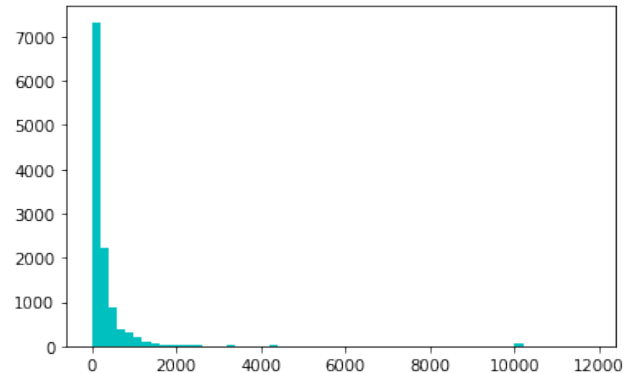
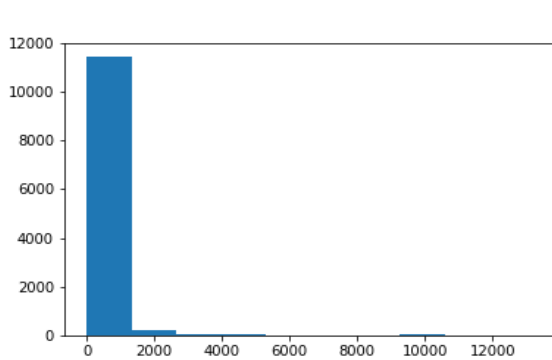
Convert to Numbers

```
1 #Method 1  
2 price_1n = price_1.astype(float)  
3 price_1n[[5, 173, 215, 400]]  
  
5          99.00  
173       2000.00  
215       1000.00  
400       1300.00  
Name: price, dtype: float64  
  
1 #Method 2  
2 price_2n = pd.to_numeric(price_1, errors = 'coerce')  
3 #transform non-numeric values into NaN  
4 price_1n.equals(price_2n)  
  
True  
  
1 df_price = pd.concat([df_full['id'], price_1n], axis = 1)  
2 df_price.head()
```

```
      id  price  
0  1078   85.0  
1  2265  225.0  
2  5245  100.0  
3  5245   99.0  
4  5760   40.0
```

Histograms

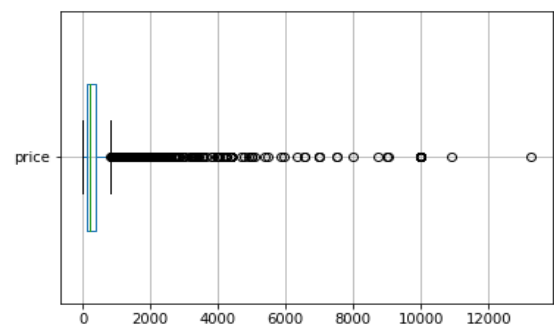
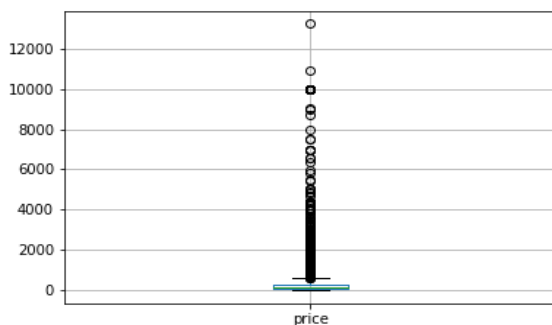
```
1 #Basic histogram
2 plt.hist(df_price.price)
```



```
1 #Customize bin widths and colors
2 bin_width = 200
3 plt.hist(df_price.price, bins = range(0, 13000, bin_width), color =
    'c')
```

Boxplots

```
1 #a basic boxplot
2 df_price.boxplot(column = 'price')
```



```
1 #A horizontal boxplot
2 df_price.boxplot(column = 'price', vert = False, widths = 0.5)
```

After Detecting Outliers

```
1 #Remove outliers: 2 equivalent ways
2 df_price1 = df_price[~df_price['price'].isin(tukey_values)]
3 df_price2 = df_price[~df_price.index.isin(tukey_indices)]
```

```
1 # Access data store
2 data_store = pd.HDFStore('property_cleaned.h5')
3 # Retrieve data using key
4 df_property = data_store['preprocessed_property']
5 data_store.close()
6
7 sizes = df_property['property_type'].value_counts()
8 sizes
```

```
House      6498
Apartment  3208
Condominium 863
Other       736
Hotel       487
```

```
Name: property_type, dtype: int64
```

```
1 df_4box = pd.merge(df_price1, df_property, on = 'id')
```

```
   id  price  property_type
0  1078    85.0           Hotel
1  2265   225.0           House
2  5245   100.0           House
3  5456    95.0           Hotel
```

Emily Zhu, PhD.

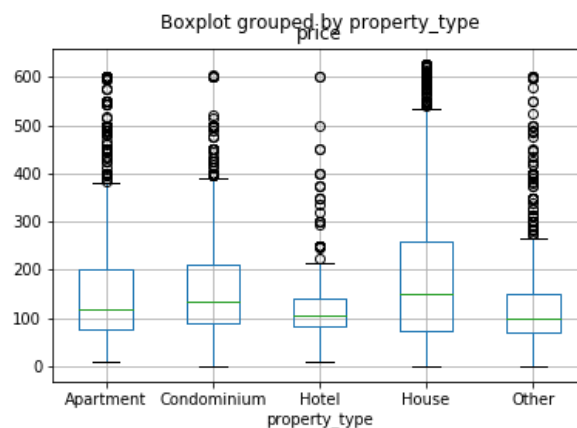
QMST5336: ANALYTICS

© 2023 Texas State University

25 / 30

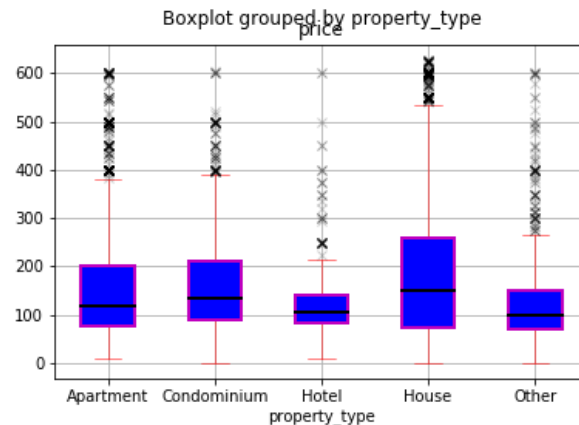
Boxplots by Property_Type

```
1 df_4box.boxplot(column = 'price', by = 'property_type')
```

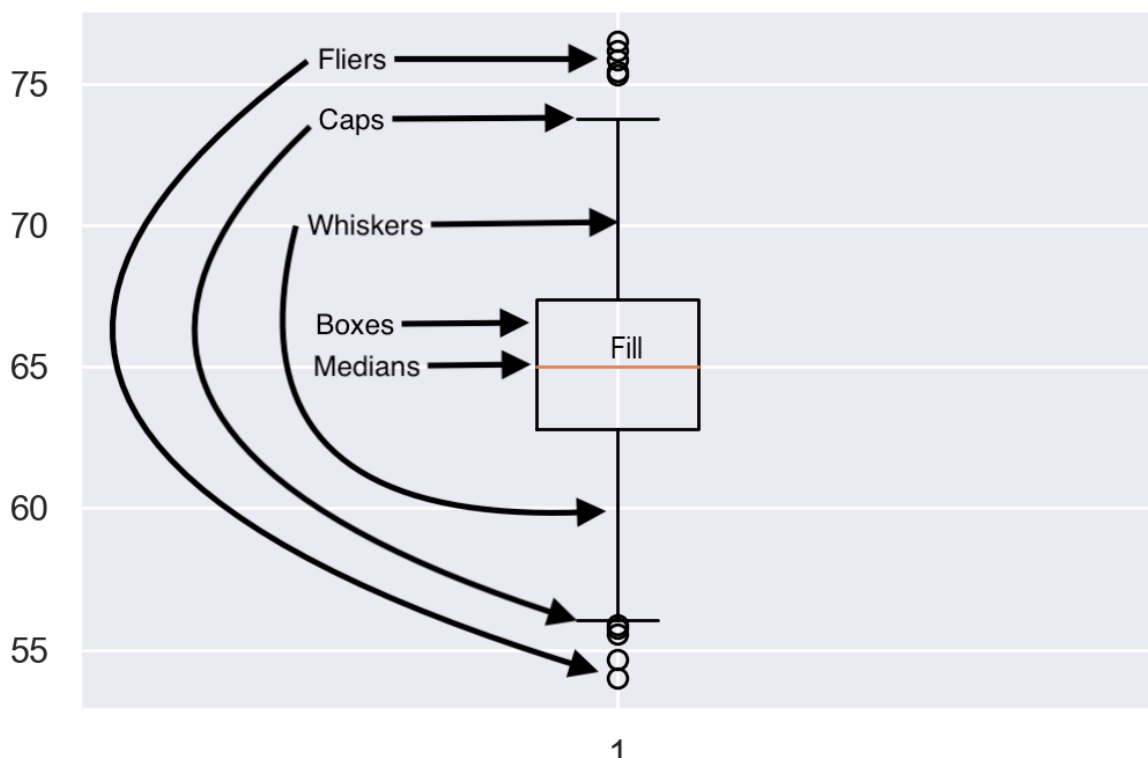


More Advanced Features of Boxplots

```
1 bp = df_4box.boxplot(column = 'price', by = 'property_type',  
2                       patch_artist = True, return_type = 'dict')  
3 # add patch_artist=True option to get fill color  
4 #return_type : {'axes', 'dict', 'both'} or None, default 'axes'  
5 #The kind of object to return. The default is axes.  
6 #'axes' returns the matplotlib axes the boxplot is drawn on.  
7 #'dict' returns a dictionary whose values are the matplotlib Lines  
8   of the boxplot.  
9 #'both' returns a namedtuple with the axes and dict.
```



Terminology: Advanced Features of Boxplots



More Advanced Features of Boxplots C'td

```
1 ## change outline color, fill color and linewidth of the boxes
2 for box in bp.price['boxes']:
3     # change outline color
4     box.set( color='m', linewidth=2)
5     # change fill color
6     box.set( facecolor = 'b' )
7
8 ## change color and linewidth of the whiskers
9 for whisker in bp.price['whiskers']:
10     whisker.set(color='r', linewidth = 0.5)
11
12 ## change color and linewidth of the caps
13 for cap in bp.price['caps']:
14     cap.set(color='r', linewidth = 0.5)
15
16 ## change color and linewidth of the medians
17 for median in bp.price['medians']:
18     median.set(color='k', linewidth=2)
19
20 ## change the style of fliers and their fill
21 for flier in bp.price['fliers']:
22     flier.set(marker='x', color='y', alpha=0.2)
23
24 plt.savefig('bp_m1.png')
```

Emily Zhu, PhD.

QMST5336: ANALYTICS

© 2023 Texas State University

29 / 30

Before Next Lecture

Sign up groups due on Sep. 13

